

**POSTS AND TELECOMMUNICATIONS INSTITUTE
OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY 1**



**SOFTWARE ARCHITECTURE
AND DESIGN**

Assignment 5

Lecturer: Tran Dinh Que

Member: B22DCCN441 – Nguyen Duc Khai

Class: E22CNPM01

Hà Nội, 2026

Table of Contents

Table of Contents	2
1. Giới thiệu	2
1.1. Bối cảnh chuyển đổi kiến trúc	2
1.2. Phân rã theo Bounded Context	3
2. Kiến trúc Hệ thống	4
2.1. Sơ đồ Tổng thể	4
2.2. Vai trò của API Gateway	4
2.3. Kiến trúc Nội bộ Dịch vụ Backend	6
3. Quản lý Dữ liệu	7
3.1. Tính đóng gói dữ liệu	7
3.2. Ưu và nhược điểm khi dùng SQLite	7
4. Giao tiếp Liên dịch vụ	7
4.1. Luồng tạo tài khoản	8
4.2. Luồng thêm hàng vào giỏ	8
4.3. Luồng thanh toán tập trung	9
5. Triển khai với Docker	10
5.1. Cấu trúc Dockerfile	10
5.2. Điều phối bằng Docker Compose	11
6. Tài liệu API	11
7. Tổng kết	12
7.1. Đánh giá kết quả đạt được	12
7.2. Bài học rút ra	13
7.3. Hướng phát triển tương lai	13

1. Giới thiệu

1.1. Bối cảnh chuyển đổi kiến trúc

Trong giai đoạn đầu phát triển, các hệ thống thương mại điện tử thường được xây dựng theo kiến trúc khối nguyên khối (Monolithic Architecture) nhằm tối ưu tốc độ triển khai và đơn giản hóa việc kiểm thử. Tuy nhiên, khi quy mô nghiệp vụ mở rộng, khối lượng người dùng tăng cao, kiến trúc Monolith bắt đầu bộc lộ những hạn chế cốt lõi:

- Khó khăn trong mở rộng (Scalability): Toàn bộ ứng dụng phải được nhân bản (scale) đồng thời, ngay cả khi nút thắt cổ chai (bottleneck) chỉ nằm ở một tính năng cụ thể (ví dụ: Service thanh toán bị quá tải trong các đợt Flash Sale).

- Rủi ro triển khai (Deployment Risk): Một thay đổi nhỏ ở mô-đun bình luận cũng có thể làm sập toàn bộ hệ thống đặt hàng nếu xảy ra lỗi không lường trước (Memory Leak, Unhandled Exception) do chia sẻ chung Resource Pool.
- Rào cản công nghệ (Technology Lock-in): Rất khó để áp dụng các công nghệ mới (như AI Recommender System) vào một hệ thống cũ kỹ mà không phá vỡ cấu trúc hiện tại.

Để giải quyết triệt để các rào cản trên, dự án BookStore đã được tái thiết kế và phân rã sang kiến trúc Microservices. Mô hình này cho phép chia nhỏ một ứng dụng công kênh thành một tập hợp các dịch vụ nhỏ gọn, độc lập, loosely coupled (liên kết lỏng lẻo) và có thể triển khai riêng biệt.

1.2. Phân rã theo Bounded Context

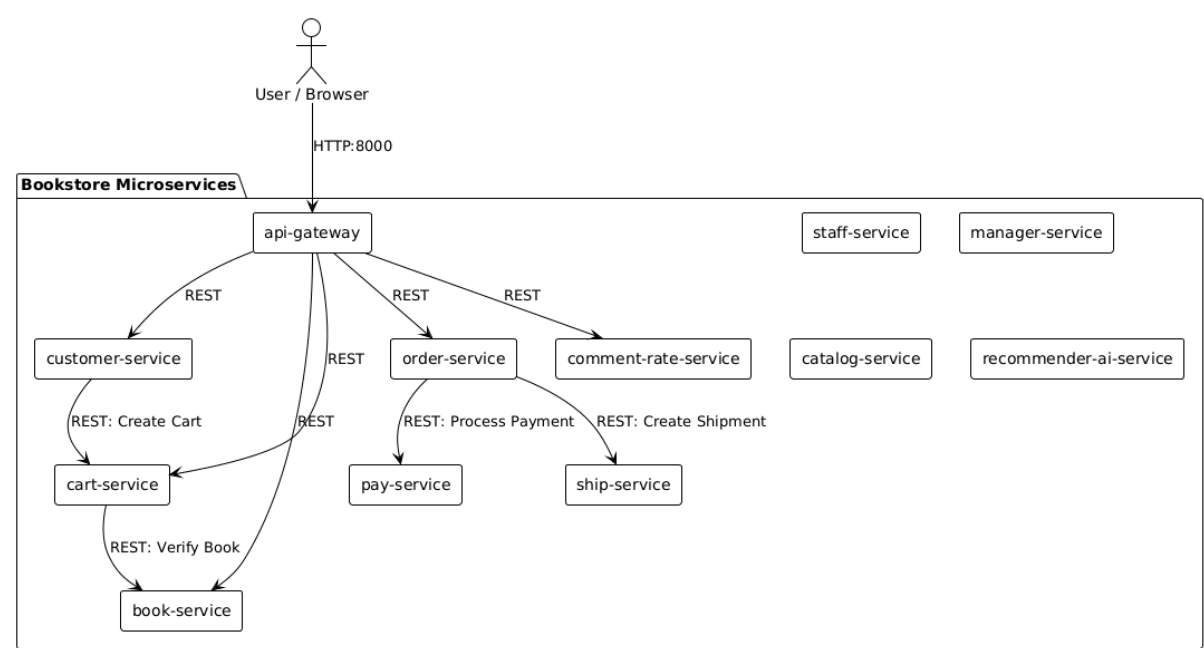
Dựa trên nguyên tắc Bounded Context của Domain-Driven Design (DDD), hệ thống BookStore được phân tách thành 12 dịch vụ cốt lõi, mỗi dịch vụ chịu trách nhiệm cho một miền nghiệp vụ duy nhất:

1. api-gateway: Điểm giao tiếp trung tâm với người dùng, xử lý UI (Bootstrap 5) và điều phối các API Call nội bộ.
2. customer-service: Quản lý vòng đời tài khoản và hồ sơ người dùng mua hàng.
3. book-service: Quản lý kho sách, thông tin ấn phẩm và số lượng tồn kho.
4. cart-service: Xử lý giỏ hàng tạm thời của người dùng trước khi tiến hành thanh toán.
5. order-service: Điều phối chu trình đặt hàng, đóng vai trò như một Saga Orchestrator thu nhỏ.
6. pay-service: Quản lý các giao dịch thanh toán (COD, thẻ tín dụng).
7. ship-service: Quản lý thông tin vận chuyển và theo dõi lộ trình đơn hàng.
8. comment-rate-service: Quản lý hệ thống đánh giá, phản hồi của người dùng về sản phẩm.
9. catalog-service: Quản lý các danh mục phân loại sách.
10. staff-service: Quản lý phân quyền và tài khoản của nhân viên điều hành.
11. manager-service: Hệ thống quản lý cấp cao, cung cấp báo cáo và thống kê tổng hợp.
12. recommender-ai-service: Dịch vụ AI chuyên biệt cung cấp các gợi ý mua hàng cá nhân hóa.

Mỗi microservice đều được đóng gói thành một Docker Container độc lập, sử dụng Django REST Framework làm nền tảng xử lý logic và cung cấp RESTful API.

2. Kiến trúc Hệ thống

2.1. Sơ đồ Tổng thể

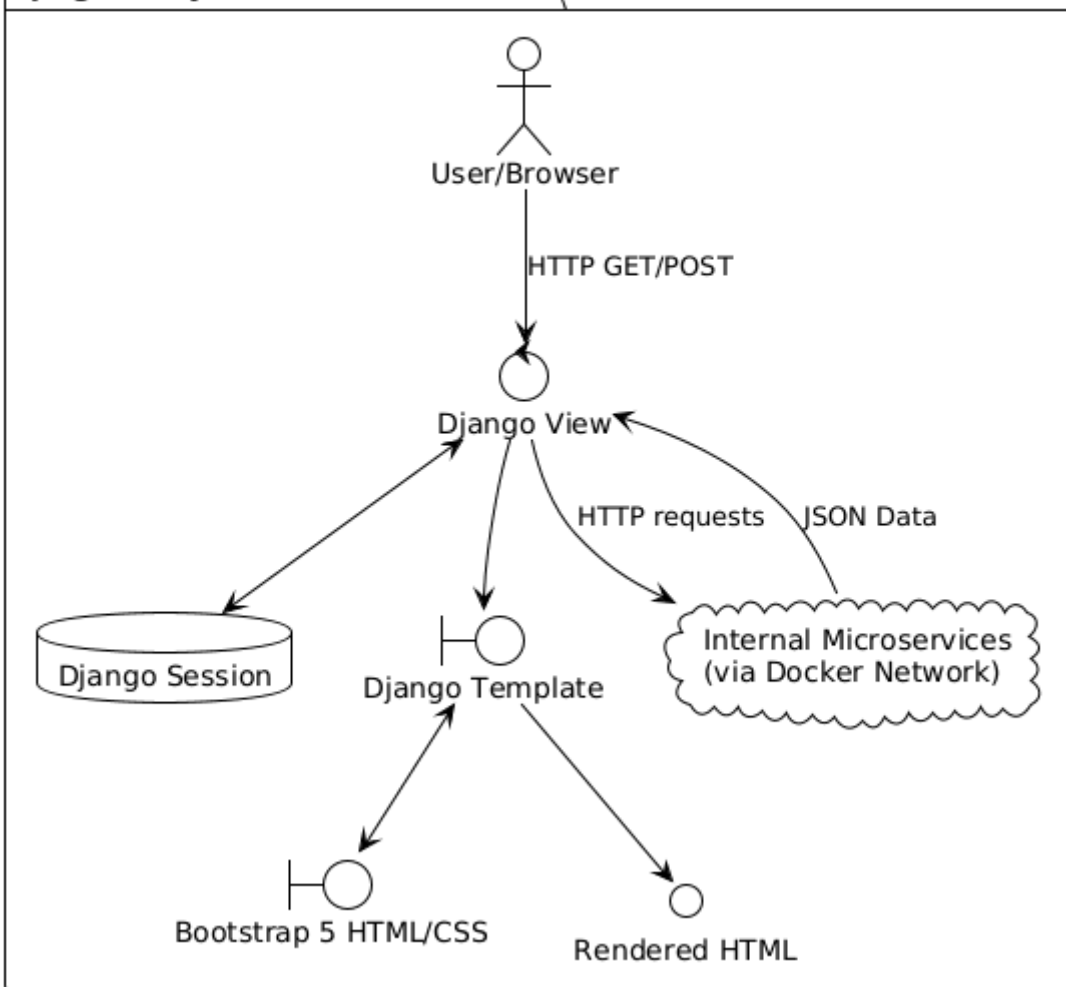


Sơ đồ 1: System Architecture Diagram

2.2. Vai trò của API Gateway

Trong hệ thống BookStore Microservices, api-gateway đóng vai trò là xương sống kiến trúc, thực thi pattern API Gateway kết hợp với Backend For Frontend (BFF).

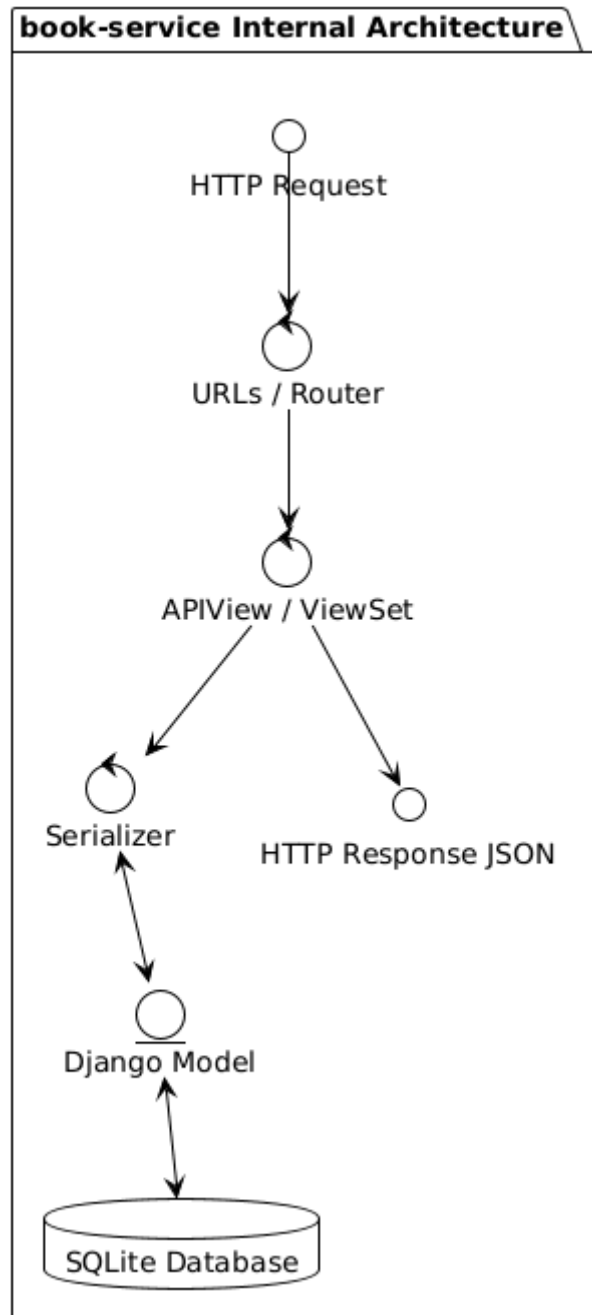
api-gateway Internal Architecture



Sơ đồ 2: Kiến trúc API Gateway

- Che giấu độ phức tạp: Người dùng cuối (Trình duyệt web/Mobile App) không cần biết sự tồn tại của 11 dịch vụ backend phía sau. Họ chỉ tương tác với duy nhất một địa chỉ HTTP (cổng 8000).
- Giao diện người dùng (UI Composition): Khác với kiến trúc API Gateway thuần túy chỉ làm nhiệm vụ Reverse Proxy (như Nginx hay Kong), api-gateway trong dự án này trực tiếp sử dụng Django Templates kết hợp Bootstrap 5 để render HTML. Các hàm xử lý trong views.py của api-gateway sẽ đóng vai trò Aggregator, gọi nhiều API từ các microservices khác nhau, tổng hợp dữ liệu (JSON) và binding vào HTML trước khi trả về cho Client.
- Quản lý phiên làm việc & Xác thực (Authentication): Gateway chịu trách nhiệm nắm giữ Session, thực thi Login/Logout và chuyển đổi danh tính người dùng (User ID) truyền xuống các dịch vụ bên dưới.

2.3. Kiến trúc Nội bộ Dịch vụ Backend



Sơ đồ 3: Internal Architecture Diagram

Mỗi microservice (ví dụ: book-service, order-service) đại diện cho một RESTful API server hoàn chỉnh, tuân thủ chặt chẽ kiến trúc MVC/MVT thông qua Django REST Framework (DRF):

- Router / URLs: Định tuyến các HTTP Request (GET, POST, PUT, DELETE) vào đúng các Endpoint xử lý.
- APIView / ViewSet: Chứa logic nghiệp vụ (Business Logic) điều hướng. Đảm nhiệm việc nhận dữ liệu đầu vào, gọi các logic xử lý chéo (ngôn ngữ Python) và quyết định HTTP Status Code trả về (200 OK, 201 Created, 400 Bad Request, 404 Not Found...).

- Serializer: Đóng vai trò Data Transfer Object (DTO) mapper. Nhiệm vụ chính là chuyển đổi dữ liệu phức tạp (như QuerySet, Model instances) thành các ràng buộc JSON gốc để trả về qua mạng, và ngược lại, validate JSON đầu vào thành các Object an toàn để lưu trữ.
- Model: Lớp ánh xạ cấu trúc dữ liệu theo chuẩn ORM (Object-Relational Mapping), tương tác trực tiếp với cơ sở dữ liệu SQLite bên dưới.

3. Quản lý Dữ liệu

Một trong những nguyên tắc tối thượng của Microservices là "Database per Service" (Mỗi dịch vụ một Cơ sở dữ liệu riêng biệt). Điều này nhằm ngăn chặn tình trạng "Tightly Coupled Data" (Ràng buộc dữ liệu chặt) — nguyên nhân chính gây ra rủi ro sập hệ thống dây chuyền ở kiến trúc Monolith.

3.1. Tính đóng gói dữ liệu

Trong hệ thống BookStore, mỗi container chứa một file db.sqlite3 hoàn toàn cô lập:

- customer-service giữ dữ liệu khách hàng.
- order-service giữ thông tin hóa đơn.
- Các dịch vụ không bao giờ được phép chia sẻ chung database hoặc dùng câu lệnh SQL JOIN xuyên dịch vụ (Cross-database JOIN).
- Nếu order-service cần biết thông tin khách hàng, nó bắt buộc phải gọi HTTP GET đến customer-service.

Sự cô lập này đảm bảo:

- Độc lập Schema: Việc thay đổi bảng Customers không làm ảnh hưởng đến khối mã nguồn của Orders.
- An toàn Dữ liệu: Nếu database của comment-rate-service bị lỗi (corrupted), các tính năng cốt lõi như mua sách và thanh toán vẫn hoạt động bình thường trên các container khác.

3.2. Ưu và nhược điểm khi dùng SQLite

Trong phạm vi học thuật và môi trường phát triển (Development), việc sử dụng SQLite mang lại sự nhẹ nhàng (lightweight) tối đa:

- Không cần cấu hình thêm các container nặng nề như PostgreSQL hay MySQL.
- File cơ sở dữ liệu nằm trực tiếp trong thư mục hệ thống (Volume mount), dễ dàng sao lưu và phục hồi chỉ bằng lệnh cp.
- Hạn chế: Không phù hợp cho môi trường Production (Sản xuất) do SQLite khóa toàn bộ file (File-level lock) trong các tác vụ ghi (Write) đồng thời, dẫn đến hiệu năng song song kém.

4. Giao tiếp Liên dịch vụ

Với nguyên tắc mỗi service có một DB riêng, các tính năng nghiệp vụ phức tạp đòi hỏi sự phối hợp của nhiều microservices. Hệ thống BookStore sử dụng Giao tiếp đồng bộ qua REST

API (Synchronous REST API Communication) thông qua thư viện requests của Python. Các lệnh gọi được thực hiện qua mạng cục bộ (Bridge Network) của Docker với tên miền nội bộ chính là tên của Container.

Dưới đây là phân tích chi tiết mã nguồn của 3 luồng giao tiếp trọng yếu nhất trong hệ thống:

4.1. Luồng tạo tài khoản

Khi một người dùng mới đăng ký tài khoản trên api-gateway, một yêu cầu POST được gửi đến customer-service. Nghiệp vụ yêu cầu: Bất cứ khi nào một khách hàng mới được tạo lập, hệ thống phải cung cấp cho họ một Giỏ hàng (Cart) trống ngay lập tức.

Tại customer-service/app/views.py:

```
def post(self, request):
    serializer = CustomerSerializer(data=request.data)
    if serializer.is_valid():
        customer = serializer.save() # Lưu khách hàng vào DB nội
bộ
        # Gọi liên dịch vụ để khởi tạo giỏ hàng
        try:
            requests.post(
                "http://cart-service:8000/carts/",
                json={"customer_id": customer.id},
                timeout=3,
            )
        except requests.exceptions.RequestException as e:
            logger.warning("Could not create cart for customer
%s", customer.id)
        return Response(CustomerSerializer(customer).data,
status=status.HTTP_201_CREATED)
```

Đánh giá kiến trúc: Đây là mô hình Eventual Consistency chủ động. customer-service thực thi lệnh gọi tới cart-service đồng bộ. Việc thiết lập timeout=3 là cơ chế bảo vệ (fail-safe) đơn giản, ngăn chặn thread bị treo nếu cart-service sập mạng.

4.2. Luồng thêm hàng vào giỏ

Khi người dùng bấm 'Thêm vào giỏ', cart-service nhận được yêu cầu lưu trữ. Tuy nhiên, trước khi ghi nhận dữ liệu CartItem, bản thân nó phải xác minh xem cuốn sách đó (Book ID) có thực sự tồn tại trong hệ thống kho (BookService) hay không.

Tại cart-service/app/views.py:

```
# Verify book exists in book-service
try:
```

```

    resp = requests.get("http://book-service:8000/books/",
timeout=5)
    resp.raise_for_status()
    book_ids = [b["id"] for b in resp.json()]
    if int(book_id) not in book_ids:
        return Response({"error": f"Book {book_id} does not
exist."}, status=status.HTTP_404_NOT_FOUND)
except requests.exceptions.RequestException as e:
    return Response({"error": "Book service unavailable."},
status=status.HTTP_503_SERVICE_UNAVAILABLE)
# Sau khi xác minh thành công, mới tiến hành lưu vào CartItem
cart, _ = Cart.objects.get_or_create(customer_id=customer_id)
item, created = CartItem.objects.get_or_create(cart=cart,
book_id=book_id)

```

Đánh giá kiến trúc: Cách triển khai này giải quyết bài toán toàn vẹn dữ liệu ngoại lai (Foreign Key Integrity) khi không còn hỗ trợ JOIN SQL giữa Table Cart và Table Book. Việc gọi API kiểm tra (Check-Then-Act) đảm bảo tính nhất quán (Consistency).

4.3. Luồng thanh toán tập trung

Order-service đóng vai trò là nhạc trưởng. Khi người dùng bấm Thanh toán, một "Saga" phức tạp được khởi chạy:

1. Tạo Object Order ban đầu ở trạng thái "Pending".
2. Khởi tạo thanh toán bên pay-service.
3. Nếu thanh toán thành công, khởi tạo giao vận bên ship-service.
4. Nếu cả hai dịch vụ đều trả về 200/201, Order mới được đánh dấu 'Confirmed'.

Tại order-service/app/views.py:

```

# 1. Khởi tạo Order cơ sở
order = Order.objects.create(customer_id=customer_id,
total_amount=total_amount)
# 2. Call pay-service
try:
    pay_resp =
requests.post("http://pay-service:8000/payments/", json={...},
timeout=5)
    pay_resp.raise_for_status()
except requests.exceptions.RequestException as e:
    order.status = "Payment Failed"
    order.save()
    return Response({"error": "Payment service failed."},
status=status.HTTP_502_BAD_GATEWAY)
# 3. Call ship-service

```

```

try:
    ship_resp =
requests.post("http://ship-service:8000/shipments/", json={...},
timeout=5)
    ship_resp.raise_for_status()
except requests.exceptions.RequestException as e:
    order.status = "Shipping Failed"
    order.save()
    return Response({"error": "Shipping service failed."},
status=status.HTTP_502_BAD_GATEWAY)
# 4. Thành công toàn vẹn
order.status = "Confirmed"
order.save()

```

Đánh giá kiến trúc: Luồng này phác họa rõ nét cách Microservices xử lý phân tán nghiệp vụ. Sự thất bại cục bộ của một API (ví dụ: ship-service crash) sẽ dẫn đến trạng thái bù đắp (Compensating Transaction) - đơn hàng lập tức bị hủy bỏ và trạng thái lỗi được ghi nhận độc lập tại order-service.

5. Triển khai với Docker

Để hiện thực hóa kiến trúc được mô tả, dự án sử dụng Docker làm công cụ đóng gói (Containerization) tiêu chuẩn cho 12 microservices.

5.1. Cấu trúc Dockerfile

Mỗi microservice đều chứa một Dockerfile chịu trách nhiệm build Image riêng rẽ. Thay vì dùng các bản phân phối Linux đầy đủ, hệ thống ưu tiên sử dụng bản python:3.10-slim tối giản:

```

# Sử dụng Python 3.10 phiên bản slim để giảm kích thước Image,
tối ưu tài nguyên
FROM python:3.10-slim
# Thiết lập biến môi trường ngăn chặn Python ghi cache (.pyc)
gây tăng dung lượng
ENV PYTHONDONTWRITEBYTECODE 1
# Bật chế độ xuất ròng (unbuffered) để theo dõi Log console hiệu
quả
ENV PYTHONUNBUFFERED 1
# Khởi tạo thư mục làm việc cố định bên trong container
WORKDIR /app
# Sao chép và phân giải thư viện phụ thuộc
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

```

```
# Triển khai mã nguồn vào Container
```

```
COPY . .
```

```
# Chạy lệnh khởi tạo server nội bộ của Django (Development Use)
```

```
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Chẩn đoán kỹ thuật:

- Tính năng python:3.10-slim cắt giảm đáng kể thời gian build CI/CD.
- Lệnh runserver 0.0.0.0:8000 cho phép ứng dụng Django chấp nhận kết nối từ mọi interface mạng bên trong không gian ảo của Docker, từ đó các container khác có thể gọi chéo nhau.

5.2. Điều phối bằng Docker Compose

Khối phổ biến nhất để quản lý môi trường Multi-container là docker-compose.yml. Thay vì chạy thủ công 12 lệnh docker run, hệ thống định nghĩa toàn bộ Topological map trong file YAML này:

- Port Mapping: Chỉ có duy nhất api-gateway được lộ diện (Expose) ra ngoài cổng 8000 của máy chủ gốc (Host machine). Các dịch vụ phía sau được gán port từ 8001 đến 8011 để mục đích debug nội bộ của nhà phát triển, nhưng hoàn toàn có thể bị gỡ bỏ trong môi trường Production (Chỉ giữ expose port ảo).
- Restart Policy: Lệnh restart: unless-stopped đảm bảo hệ thống tự phục hồi nếu một container bị Crash do Out-of-Memory hoặc máy chủ khởi động lại.
- Mạng cục bộ (Bridge Network): Tất cả 12 services đều được gán chung vào một Network do người dùng định nghĩa networks: [bookstore-net]. Docker tự phát động tính năng DNS Resolution nội bộ, cho phép api-gateway gọi http://catalog-service:8000/ thay vì phải quan tâm mớ hỗn độn địa chỉ IP động (172.x.x.x).

6. Tài liệu API

Mặc dù hệ thống được chia làm 12 dịch vụ, cốt lõi giao tiếp vẫn nằm ở các RESTful Endpoints. Dưới đây là bảng trích xuất các API trọng yếu nhất đóng vai trò giao diện (Interface) cho hệ thống BookStore.

Dịch vụ (Service)	Phương thức	Đường dẫn API (Endpoint URL)	Chức năng (Description)
api-gateway	GET	/	Trả về giao diện trang chủ Bootstrap hiển thị danh sách Sách.
api-gateway	POST	/register/	Tiếp nhận Form HTML, gọi customer-service tạo tài khoản.

Dịch vụ (Service)	Phương thức	Đường dẫn API (Endpoint URL)	Chức năng (Description)
customer-service	POST	/customers/	Khởi tạo Record User. Tự động trigger cart-service tạo Cart rỗng.
customer-service	GET	/customers/{id}/	Truy xuất thông tin người mua hàng.
book-service	GET	/books/	Liệt kê toàn bộ sách hiện hành trong kho quản lý.
book-service	POST	/books/	Thêm đầu sách mới (Chỉ dành cho quyền Staff/Admin).
cart-service	GET	/carts/{customer_id}/	Lấy dữ liệu Giỏ hàng cá nhân hóa dựa trên Session User.
cart-service	POST	/cart-items/	Validate Book ID hợp lệ và đưa ấn phẩm vào giỏ.
cart-service	PUT	/cart-items/{item_id}/	Cập nhật số lượng đầu ấn trong giỏ hàng (+, -).
order-service	POST	/orders/	Nhận lệnh "Checkout", lưu Order và trigger Saga phân tán.
pay-service	POST	/payments/	Sinh giả lập giao dịch thanh toán (Mock-up Payment Gateway).
ship-service	POST	/shipments/	Sinh lộ trình điều vận hàng hóa ảo.
comment-rate-service	POST	/reviews/	Ghi nhận Rate sao và Feedback từ người dùng.

7. Tổng kết

7.1. Đánh giá kết quả đạt được

Dự án "Phân rã hệ thống BookStore Monolith sang kiến trúc Microservices" đã hoàn thành mục tiêu tái cấu trúc một ứng dụng Django công kênh thành 12 Container chuyên biệt.

- Ưu điểm rõ rệt nhất là khả năng chia tách (Decoupling) sự phức tạp của cơ sở dữ liệu. Nhờ vào việc định nghĩa rõ ràng biên giới (Bounded Contexts), việc bổ sung một tính năng (Ví dụ: Thêm AI Recommender) không đòi hỏi việc viết lại toàn bộ mã nguồn như trước đây.

- API Gateway đã chứng minh được hiệu quả trong việc cung cấp một Single Entry Point mượt mà cho trải nghiệm người dùng cuối trên nền Web Browser truyền thống (Bootstrap 5).

7.2. Bài học rút ra

- Cái giá phải trả của Microservices: Độ trễ mạng nội bộ Docker (Network Latency) lớn hơn rất nhiều so với hàm gọi trực tiếp (Method call) trong Monolith. Các hàm xử lý giao dịch tại lớp Controller/View bị kéo dẫn thời gian.
- Thử thách quản trị lỗi: Khi một tiến trình Saga thất bại một nửa (Ví dụ: Pay thành công, Ship thất bại), dòng lệnh logic bù trừ (Compensating logic) phức tạp hơn hàm khôi phục Transaction (Rollback) của RDBMS truyền thống.

7.3. Hướng phát triển tương lai

Để tiến tới một hệ thống chuẩn Công nghiệp (Enterprise-grade), kiến trúc cần nâng cấp các khiếm khuyết trên cơ sở:

1. Chuyển đổi giao tiếp sang Kafka/RabbitMQ: Giảm phụ thuộc vào kết nối Synchronous HTTP requests (nguy cơ sụp đổ dây chuyền Cascading Failure), định hình lại luồng kiến trúc sang Giao tiếp Bất đồng bộ (Event-driven Architecture).
2. Sử dụng GraphQL tại API Gateway: Thay cho việc phải gọi nhiều endpoint REST rời rạc (Book, Rate, Catalog) ở Gateway để ghép thành một trang HTML (Over-fetching), hệ thống Backend-For-Frontend cần tích hợp GraphQL để tối ưu truy vấn dữ liệu theo cấu trúc động.
3. Database Migration: Từ bỏ SQLite, chuyển sang PostgreSQL tập trung cho từng Container (Cơ sở chung về SQL DBMS Engine, tách riêng Schema Authorization cho từng Microservice).